

Platforms & Limitations

Feature matrix (by platform)

Platform	Bluetooth MIDI	USB MIDI	Network MIDI (RTP-MIDI)	Nearby Connections MIDI	Inter-App MIDI	USB MIDI 2.0	Network MIDI 2.0 (UDP MIDI 2.0)	Scriptable Audio (optional)
iOS	○	○	○	○	○	○	△ (experimental)	○ *
Android	○	○	△ (experimental)	○	○	○	△ (experimental)	○ *
Universal Windows Platform	-	○	△ (experimental)	-	○	△ (limited)	△ (experimental)	○ *
Standalone macOS / Unity Editor macOS	○	○	○	○	○	○	△ (experimental)	○ *
Standalone Linux / Unity Editor Linux	○	○	△ (experimental)	-	○	○	△ (experimental)	○ *
Standalone Windows / Unity Editor Windows	-	○	△ (experimental)	-	○	△ (limited)	△ (experimental)	○ *
WebGL	○	○	-	-	-	-	-	-

Legend:

- ○ supported
- △ supported but experimental / limited
- ○ not supported

* Scriptable Audio integration requires **Unity 6000.3+**, scripting define `FEATURE_SCRIPTABLE_AUDIO`, and a non-WebGL build target. See [Build PostProcessing](#).

Limitations / requirements

Android

- USB MIDI: API Level 12 (Android 3.1) or above.
- Bluetooth MIDI: API Level 18 (Android 4.3) or above.
- Inter-App MIDI: API Level 23 (Android 6.0) or above.
- MIDI 2.0: API Level 23 (Android 6.0) or above.
- If you build with **Mono**:
 - latency issues may occur
 - may only support `armeabi-v7a`
 - Recommended: use **IL2CPP**
`Project Settings > Player > Configuration > Scripting Backend`
- Nearby Connections MIDI:
 - requires API Level 28 (Android 9) or above
 - and should be compiled with API Level 33 (Android 13) or above

Bluetooth MIDI Peripheral mode (Android only)

In addition to acting as a BLE MIDI **Central** (scanning/connecting to devices), Android can also act as a BLE MIDI **Peripheral** (advertise your app as a MIDI device).

Notes:

- This is Android-only.
- See [MIDI 1.0 \(MidiManager\)](#) for the API entry points.
- Platform permissions and Bluetooth state still apply (see [Build PostProcessing](#)).

Meta Quest (Oculus Quest)

Meta Quest devices run Android, so the **Android** requirements above apply.

Practical notes (mirrors the manual):

- **USB MIDI** on Quest may require enabling a USB device intent filter during Android build postprocessing.
 - See: [Build PostProcessing & Scripting Define Symbols](#) (Quest USB MIDI section)
- **Bluetooth MIDI** device discovery/pairing can use Android's Companion Device workflow.
 - Enable scripting define symbol: `FEATURE_ANDROID_COMPANION_DEVICE`
 - On Unity 6+ with **GameActivity** as Application Entry Point, the post-process selects `BleMidiUnityGamePlayerActivity` (see [Build PostProcessing](#))
 - See: [Build PostProcessing & Scripting Define Symbols](#)

iOS / macOS

- Supported iOS: 12.0 or above.

- Bluetooth MIDI: Central mode only.

UWP

- Supported UWP target version (MIDI 1.0): 10.0.10240.0 or above.
- MIDI 2.0 (USB): **limited support** (`UwpMidi2Plugin` / `Midi2Plugin.Uwp.cs`) via `InitializeMidi2`.
 - Runtime requirements:
 - OS: **Windows 11 24H2 or later** (build **26100+**, including 25H2 / 26H1) with Windows MIDI Services enabled
 - Separate install of **Windows MIDI Services SDK Runtime and Tools** on the device ([get-latest](#) or `winget install Microsoft.WindowsMIDIServicesSDK`)
 - Architecture: **x64 or ARM64** (x86 not supported)
 - Reference `Assets/MIDI/Plugins/WSA/Microsoft.Windows.Devices.Midi2.winmd` (native WinRT; do not use NetProjection)
 - **UWP player builds only** (`UNITY_WSA && !UNITY_EDITOR`). Does not run in the Unity Editor or Standalone Windows
 - On Windows 10, Windows 11 23H2 or earlier, or when MIDI Services / SDK Runtime is missing, initialization is skipped with a warning log and the completion callback is still invoked (no crash)
 - MIDI 1.0 (`WindowsMidiPlugin`) and MIDI 2.0 (`UwpMidi2Plugin`) are separate backends, so the same physical device may appear under **both** APIs with different ID spaces (classic MIDI1 IDs vs Windows MIDI Services `EndpointDeviceId`). Choose MIDI1-only or MIDI2-only as needed
 - Hot-plug is handled by the Watcher. There is no dedicated UWP Suspend / `OnApplicationPause` re-init; after a long suspend, prefer `TerminateMidi2` then `InitializeMidi2`
 - UWP builds also initialize `UdpMidi2Plugin` (experimental) alongside USB MIDI 2.0
- Bluetooth MIDI is not supported.
- RTP-MIDI requires enabling capability:
 - `Project Settings > Player > Capabilities > PrivateNetworkClientServer`

Windows (Standalone / Unity Editor)

- Bluetooth MIDI is not supported.
- MIDI 2.0 (USB): **limited support** (`WindowsMidi2Plugin` / `Midi2Plugin.Windows.cs`) via `InitializeMidi2` and native `Midi2Native.dll` (**Method C** — C++/WinRT).
 - Runtime requirements:
 - OS: **Windows 11 24H2 or later** (build **26100+**, including 25H2 / 26H1) with Windows MIDI Services enabled
 - Separate install of **Windows MIDI Services SDK Runtime and Tools** on the device ([get-latest](#) or `winget install Microsoft.WindowsMIDIServicesSDK`)
 - Architecture: **x64 or ARM64** (x86 not supported). Unity Editor Play Mode uses the **x64** DLL
 - Native plugins: `Assets/MIDI/Plugins/Windows/x86_64/Midi2Native.dll` and `.../ARM64/Midi2Native.dll` . Do **not** add `WinRT.Runtime` , NetProjection, or a Standalone copy of `Microsoft.Windows.Devices.Midi2.winmd` to managed `Assets`

- **Standalone Windows and Unity Editor Windows** (`UNITY_EDITOR_WIN` || `(UNITY_STANDALONE_WIN && !UNITY_EDITOR)`). UWP USB MIDI 2.0 remains **Method B** (`UwpMidi2Plugin` + WSA winmd) — a different backend; do not enable the WSA winmd for Standalone
 - On Windows 10, Windows 11 23H2 or earlier, or when MIDI Services / SDK Runtime is missing, initialization is skipped with a warning log and the completion callback is still invoked (no crash)
 - MIDI 1.0 (`WindowsMidiPlugin` / WinMM) and MIDI 2.0 (`WindowsMidi2Plugin`) are separate backends, so the same physical device may appear under **both** APIs with different ID spaces (WinMM IDs vs Windows MIDI Services `EndpointDeviceId`). Choose MIDI1-only or MIDI2-only as needed
 - Hot-plug is handled by the Watcher. Editor Play Mode exit calls `TerminateMidi` via `PlayModeStateChanged` (native SDK shutdown is refcount-safe)
 - Standalone / Editor builds also initialize `UdpMidi2Plugin` (experimental) alongside USB MIDI 2.0 when using `InitializeMidi2`

Linux

- When MIDI 1 and MIDI 2 coexist on one USB device, only the MIDI 2 port may be found.

WebGL

- Device support depends on OS/browser WebMIDI environment.
- WebGL can't use raw UDP/TCP sockets, so RTP-MIDI / UDP MIDI 2.0 are unavailable.
- WebGL may have restricted `UnityWebRequest` access to other servers; use `StreamingAssets` for `.mid` and other content.
- WebGL cannot handle USB MIDI 2.0 devices nor network MIDI 2.0; MIDI 2.0 runtime functions are not available except clip file read/write.
- WebGL template may need to expose `unityInstance` (see [Transports & Platform Notes](#)).
- **Scriptable Audio integration is not compiled on WebGL** (`jp.kshoji.midi.scriptableaudio` is excluded from the WebGL platform in `asmdef`). `ScriptableAudioUtility.IsAvailable` is `false` ; no integration types are referenced at runtime.
- **Maestro / MPTK on WebGL (optional):** Maestro 2.16+ documents official WebGL support. Expect **Core player legacy** (`MPTK_Core` forced off on WebGL). Prefer content in `StreamingAssets` or a **CORS-compliant** host for `MidiExternalPlayer` / remote SoundFonts. `MPTKWriter` on WebGL is direct-play oriented. **MPTK available ≠ every plugin integration available** (Scriptable Audio / Network MIDI / MIDI 2.0 runtime remain limited as above). See [Maestro / MPTK Integration](#).

Scriptable Audio Pipeline (optional integration)

Requirement	Value
Unity version	6000.3 LTS or newer
Scripting define	<code>FEATURE_SCRIPTABLE_AUDIO</code>
Packages	<code>com.unity.burst</code> , <code>com.unity.collections</code> (when using the full repo)
WebGL	Not supported — integration <code>asmdef</code> excluded; source guarded by <code>#if !UNITY_WEBGL</code>
Symbol off / Unity < 6.3	Integration not compiled; core MIDI plugin unchanged

When requirements are not met, the project should still compile without errors. See [Build PostProcessing](#).

Chunity / Chunity Scriptable Generator (optional integration)

Item	Details
Chunity runtime	Not bundled ; installed separately by the user
Scripting define	<code>FEATURE_CHUNITY</code> (required). Additionally <code>FEATURE_CHUNITY_SCRIPTABLE_AUDIO</code> when using the Generator
Packages (Generator)	<code>com.unity.collections</code> (<code>asmdef</code> references <code>Unity.Collections</code>)
Generator patch	Minimal <code>useBuiltInAudioFilter</code> patch to Chunity itself (Chunity Optional patch guide)
WebGL	Follows Chunity's own constraints; the Generator is unsupported and falls back to <code>FilterRead</code> / <code>WebChuck</code>
Unity version (Generator)	6000.3+ expected (same line as the built-in Scriptable Audio integration)

See [Unity Ecosystem Integrations — Chunity](#) and [Build PostProcessing](#) for details.